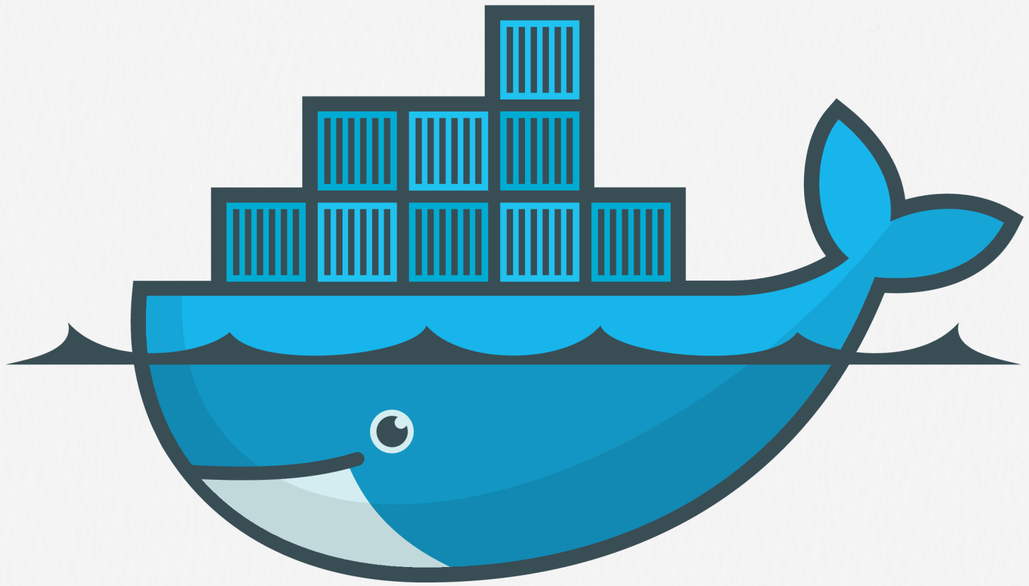
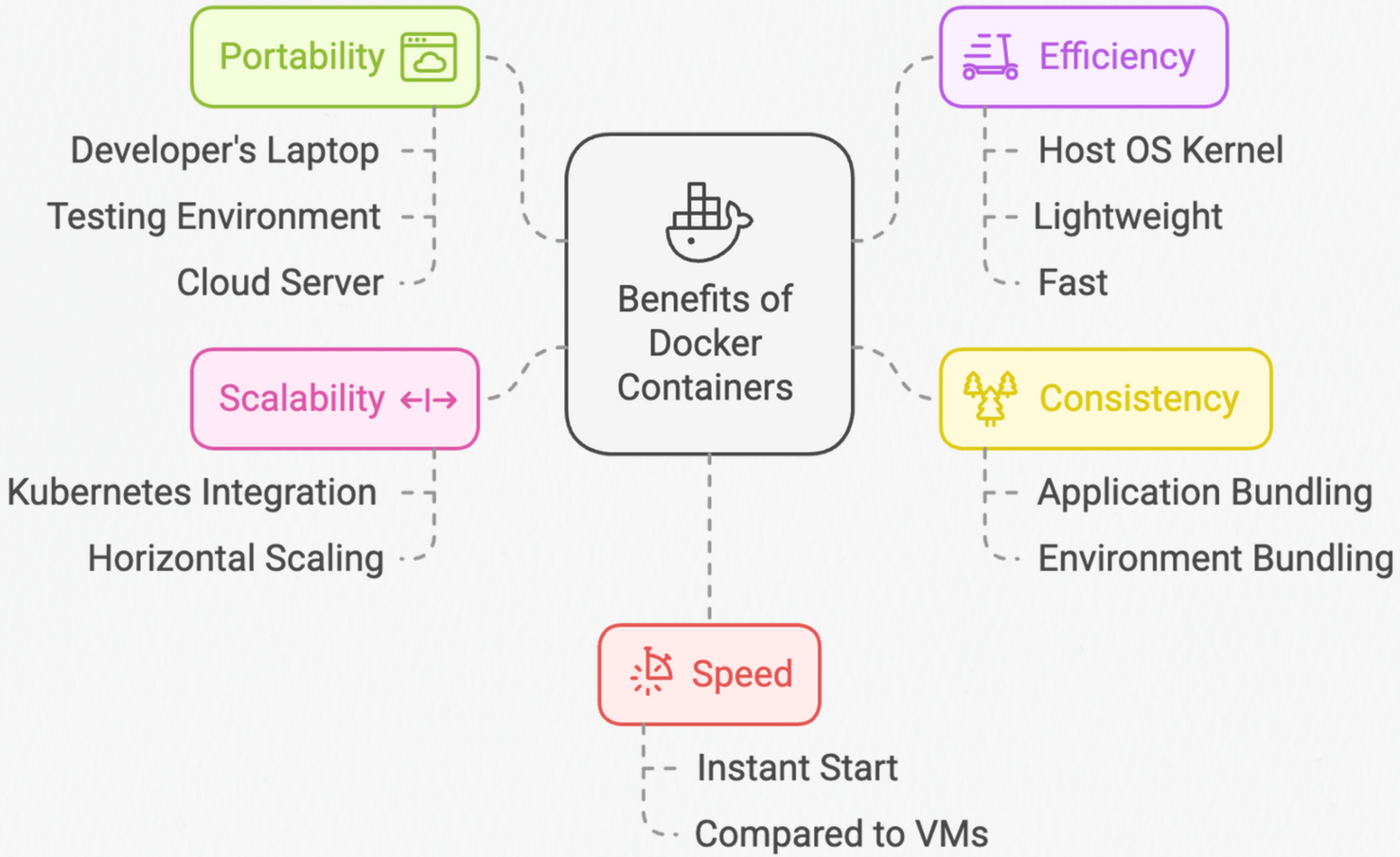
What is Docker?

- Docker is an open-source platform designed to **help developers build, ship, and run applications in a lightweight, portable, and consistent manner using containers**.
- This approach ensures that the application runs **consistently across different environments**, such as **development**, **staging**, and **production**



Key Benefits of Docker

SILICON VALLEY ENGINEER



Key Docker Components

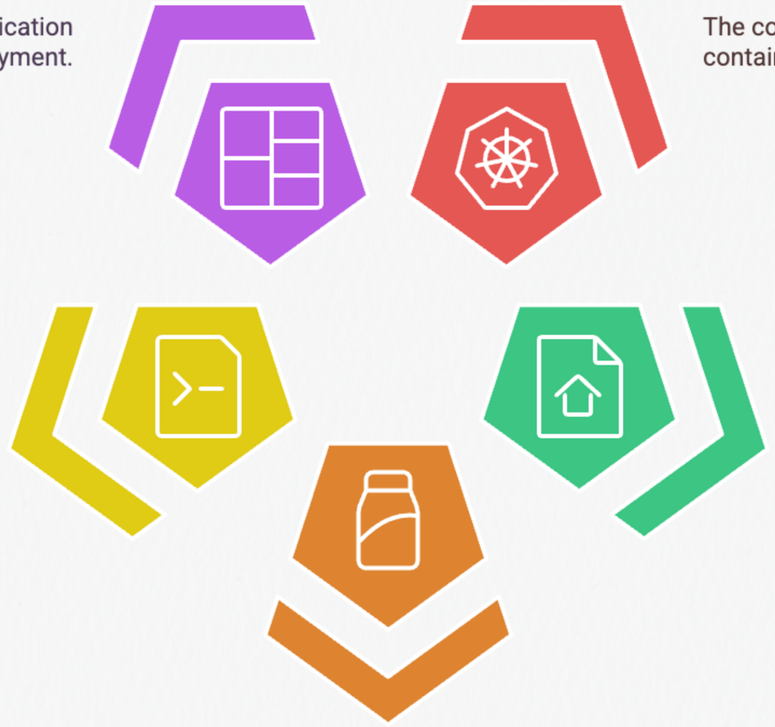
SILICON VALLEY ENGINEER

Docker Compose

Facilitates multi-container application deployment.

Docker Engine

The core component managing container operations.



Dockerfile

Scripts outlining the creation of images.

Docker Images

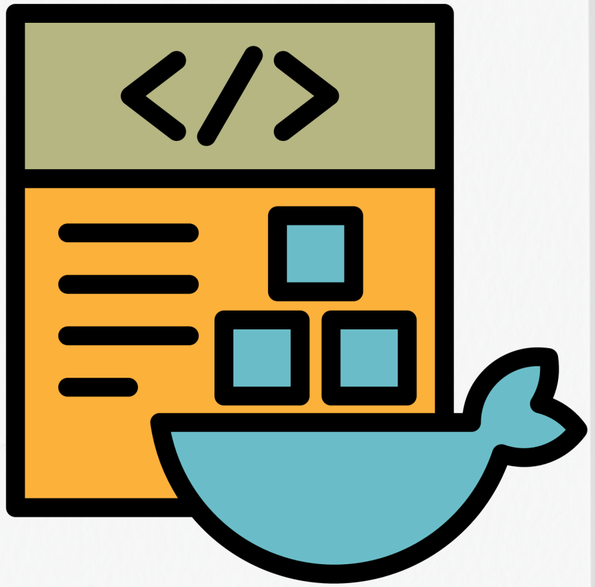
Templates that encapsulate applications and dependencies.

Docker Containers

Active instances of images, isolated yet communicative.

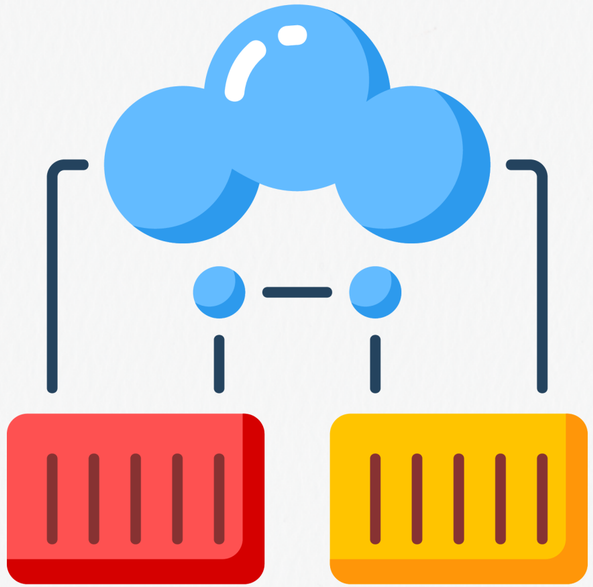
Key Docker Components

- **Docker Engine**
  - Core part of Docker that creates, runs, and manages containers.
  - Consists of
    - **Server (dockerd)**: The daemon that runs the containers.
    - **REST API**: Interfaces with the daemon to control Docker.
    - **CLI (Command-Line Interface)**: A tool for managing containers.
- **Docker Images**
  - **Read-only** templates containing the application and its dependencies.
  - Can be **built from a Dockerfile**, which specifies the steps to create the image.
- **Docker Containers**
  - **Runtime instances** of Docker images.
  - Containers are isolated but can **communicate with each other**.



Key Docker Components

- **Dockerfile**
  - A **script** that defines how to build a Docker image.
  - Includes instructions like which base image to use, copying files, and running commands.
- **Docker Compose**
  - A **tool for defining and running multi-container Docker applications**.
  - Uses a **docker-compose.yml** file to specify the configuration.
- **Docker Hub**
  - A cloud-based **registry for sharing Docker images**.
  - Developers can pull pre-built images or push their own.



Docker Use Cases

- **Development and Testing**
  - Set up consistent development environments.
  - Quickly test applications in isolated containers.
- **Microservices Architecture**
  - Deploy and manage services independently in containers.
- **Continuous Integration/Continuous Deployment (CI/CD)**
  - Automate testing and deployment pipelines.
- **Hybrid Cloud and Multi-Cloud Deployments**
  - Run the same containerized application across different environments.
- **Big Data and Machine Learning**
  - Simplify the setup of environments for data processing frameworks like Apache Spark.

Example of Docker Workflow

- Write a Dockerfile to define the application environment.
- Build a Docker image using the command
  - **docker build -t my-app .**
- Run a container from the image
  - **docker run -d -p 8080:80 my-app**
- Push the image to Docker Hub or another registry
  - **docker push my-app**
- Deploy the container in production or an orchestrated cluster.